

05

/ 13 LECTURES

Graph Algorithms for Networking

Networks are graphs. Learn BFS, DFS and Dijkstra on a real network — and see exactly which routing protocol is powered by each algorithm.

Graph Representation

Adjacency List · Matrix

BFS — Queue

DFS — Stack

Cycle Detection

Dijkstra

Shortest Path

RIP · OSPF mapping

WHAT THIS LECTURE GIVES YOU

A complete, traced tour of the three algorithms behind Internet routing: representing a network as a weighted graph, BFS for fewest hops (→ RIP), DFS for topology exploration and loop detection, and Dijkstra for least-cost paths (→ OSPF) — with Java and C++ code, step-by-step traces, and a full Dijkstra table you can reproduce in the exam.

STUDY-READY EDITION

CONCEPTS · CODE · TRACES · EXAM PREP

LEARNING OBJECTIVES — AFTER THIS LECTURE YOU CAN

- Model a network as a **weighted graph** and choose the right representation
- Implement and **trace BFS** for hop-count shortest paths
- Implement and **trace DFS** for topology exploration and cycle detection
- Apply **Dijkstra** for least-cost routing and reconstruct the path

Graphs for Networking



1 Networks as Graphs

A graph $G = (V, E)$: **V** = routers (vertices/nodes), **E** = links (edges), and each edge carries a **weight** = link cost (latency, bandwidth-derived cost, or hop cost). Our running example — a 5-city network:

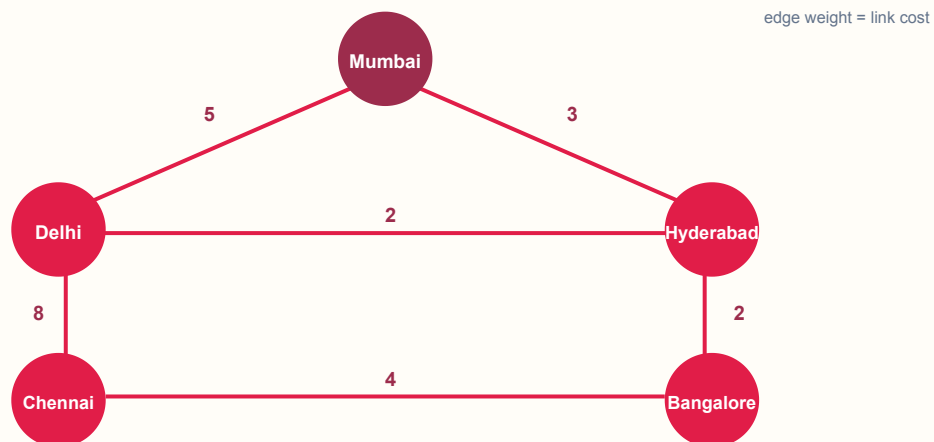


Figure 1. The running example graph used for every trace in this lecture.

ADJACENCY LIST – SPARSE GRAPHS

JAVA

```
// Edge = (neighbour, cost). Map from node ->
// its edges.
Map<String, List<Edge>> graph = new HashMap<>
();
graph.put("Mumbai", List.of(new
Edge("Delhi", 5), new Edge("Hyderabad", 3)));
graph.put("Delhi", List.of(new
Edge("Mumbai", 5), new Edge("Hyderabad", 2),
new
Edge("Chennai", 8)));
graph.put("Hyderabad", List.of(new
Edge("Mumbai", 3), new Edge("Delhi", 2),
new
Edge("Bangalore", 2)));
graph.put("Chennai", List.of(new
Edge("Delhi", 8), new Edge("Bangalore", 4)));
graph.put("Bangalore", List.of(new
Edge("Hyderabad", 2), new Edge("Chennai",
4)));
// Edge is a tiny record: record Edge(String
to, int w) {}
```

ADJACENCY LIST – SPARSE GRAPHS

C++

```
// Edge = (neighbour, cost). Map from node ->
// its edges.
map<string, vector<pair<string,int>>> graph;
graph["Mumbai"] = {{ "Delhi", 5},
{"Hyderabad", 3}};
graph["Delhi"] = {{ "Mumbai", 5},
{"Hyderabad", 2}, {"Chennai", 8}};
graph["Hyderabad"] = {{ "Mumbai", 3},
{"Delhi", 2}, {"Bangalore", 2}};
graph["Chennai"] = {{ "Delhi", 8},
{"Bangalore", 4}};
graph["Bangalore"] = {{ "Hyderabad", 2},
{"Chennai", 4}};
```

ADJACENCY MATRIX – DENSE GRAPHS

JAVA

```
//      M    D    H    C    B
int INF = Integer.MAX_VALUE;
// "no direct link"
int[][] m = {
    { 0,  5,  3, INF, INF }, // Mumbai
    { 5,  0,  2,  8, INF }, // Delhi
    { 3,  2,  0, INF,  2 }, // Hyderabad
    { INF, 8, INF,  0,  4 }, // Chennai
    { INF, INF, 2,  4,  0 }  // Bangalore
};
```

ADJACENCY MATRIX – DENSE GRAPHS

C++

```
//      M    D    H    C    B
const int INF = 1e9;
// "no direct link"
vector<vector<int>> m = {
    { 0,  5,  3, INF, INF }, // Mumbai
    { 5,  0,  2,  8, INF }, // Delhi
    { 3,  2,  0, INF,  2 }, // Hyderabad
    { INF, 8, INF,  0,  4 }, // Chennai
    { INF, INF, 2,  4,  0 }  // Bangalore
};
```

CHOICE The Internet is **sparse** (each router connects to a few others) → adjacency **list** wins: $O(V+E)$ space vs $O(V^2)$ for the matrix.

2 BFS — Breadth-First Search

What BFS finds

The **minimum hop-count** path from source to every reachable node. Uses a **queue (FIFO)**; explores layer by layer: 1 hop away, then 2 hops, then 3...

Limitation

BFS minimises **hops, not cost**. A 1-hop path costing 100 beats a 3-hop path costing 3 — wrong for cost-based routing. For cost, use **Dijkstra** (§4).

BFS WITH PATH TRACKING

JAVA

```

static List<String> bfsPath(Map<String, List<Edge>> g,
                          String start, String end) {
    Deque<List<String>> queue = new ArrayDeque<>();
    Set<String> visited = new HashSet<>();
    queue.add(List.of(start)); visited.add(start);
    while (!queue.isEmpty()) {
        List<String> path = queue.poll(); // FIFO – the queue is the whole trick
        String node = path.get(path.size() - 1);
        if (node.equals(end)) return path; // first visit = fewest hops
        for (Edge e : g.get(node)) {
            if (visited.add(e.to())) {
                List<String> next = new ArrayList<>(path);
                next.add(e.to());
                queue.add(next); // carry the path alongside
            }
        }
    }
    return null; // end unreachable from start
}

```

BFS WITH PATH TRACKING

C++

```

vector<string> bfsPath(map<string, vector<pair<string,int>>>& g,
                     string start, string end) {
    queue<vector<string>> q;
    set<string> visited;
    q.push({start}); visited.insert(start);
    while (!q.empty()) {
        auto path = q.front(); q.pop(); // FIFO – the queue is the whole trick
        string node = path.back();
        if (node == end) return path; // first visit = fewest hops
        for (auto& [nb, w] : g[node]) {
            if (!visited.count(nb)) {
                visited.insert(nb);
                auto next = path; next.push_back(nb);
                q.push(next); // carry the path alongside
            }
        }
    }
    return {}; // end unreachable from start
}

```

Trace — BFS from Mumbai to Bangalore

Step	Visit	Queue after	Notes
init	—	[Mumbai]	visited = {Mumbai}
1	Mumbai	[Delhi, Hyderabad]	adds both neighbours
2	Delhi	[Hyderabad, Chennai]	Hyderabad already visited
3	Hyderabad	[Chennai, Bangalore]	adds Bangalore
4	Chennai	[Bangalore]	Bangalore already queued
5	Bangalore	[]	FOUND! Path: Mumbai → Hyderabad → Bangalore (2 hops)

3 DFS — Depth-First Search

DFS explores **as deep as possible** along one branch before backtracking — a **stack** (or recursion) instead of a queue. Finds *a* path, not necessarily the shortest. $O(V+E)$.

DFS — ITERATIVE + CYCLE DETECTION

JAVA

```
static List<String> dfsIterative(Map<String, List<Edge>> g, String start) {
    Deque<String> stack = new ArrayDeque<>(List.of(start));
    Set<String> visited = new HashSet<>();
    List<String> order = new ArrayList<>();
    while (!stack.isEmpty()) {
        String node = stack.pop();           // LIFO — go deep first
        if (visited.add(node)) {
            order.add(node);
            for (Edge e : g.get(node))
                if (!visited.contains(e.to())) stack.push(e.to());
        }
    }
    return order;
}

static boolean hasCycle(String node, Set<String> visited, Set<String> recStack,
                        Map<String, List<Edge>> g) {
    visited.add(node); recStack.add(node);    // on the current path
    for (Edge e : g.get(node)) {
        if (!visited.contains(e.to())) {
            if (hasCycle(e.to(), visited, recStack, g)) return true;
        } else if (recStack.contains(e.to())) {
            return true;                      // back edge = cycle!
        }
    }
    recStack.remove(node);                    // leave the current path
    return false;
}
```

DFS — ITERATIVE + CYCLE DETECTION

C++

```

vector<string> dfsIterative(map<string, vector<pair<string,int>>>& g, string start) {
    stack<string> st; st.push(start);
    set<string> visited; vector<string> order;
    while (!st.empty()) {
        string node = st.top(); st.pop();        // LIFO – go deep first
        if (!visited.count(node)) {
            visited.insert(node); order.push_back(node);
            for (auto& [nb, w] : g[node])
                if (!visited.count(nb)) st.push(nb);
        }
    }
    return order;
}

bool hasCycle(const string& node, set<string>& visited, set<string>& recStack,
              map<string, vector<pair<string,int>>>& g) {
    visited.insert(node); recStack.insert(node);    // on the current path
    for (auto& [nb, w] : g[node]) {
        if (!visited.count(nb)) {
            if (hasCycle(nb, visited, recStack, g)) return true;
        } else if (recStack.count(nb)) {
            return true;                          // back edge = cycle!
        }
    }
    recStack.erase(node);                          // leave the current path
    return false;
}

```

Trace — DFS from Mumbai

```

Visit Mumbai → push Delhi, Hyderabad
Pop Hyderabad → push Bangalore
Pop Bangalore → push Chennai
Pop Chennai → Delhi already seen
Pop Delhi → all neighbours visited
Order: Mumbai → Hyderabad → Bangalore →
Chennai → Delhi

```

Network uses of DFS

- **Topology discovery** — walk every reachable router
- **Cycle detection** — a cycle in the routing graph = a potential **routing loop** (packets circling forever)
- **Spanning-tree construction** (preview of Lecture 6 / STP)

4 Dijkstra — Minimum-Cost Paths

Idea

Greedy: always expand the cheapest unfinished node; relax its neighbours. A **min-heap** serves the cheapest node fast.

Guarantee

Optimal costs to ALL nodes — but only when **all edge weights ≥ 0** (why: Lecture 6).

Complexity

$O((V+E) \log V)$ with a binary heap — the price of the priority queue, worth it for correct costs.

```

record PQ(int cost, String node) {}

static Map<String, Integer> dijkstra(Map<String, List<Edge>> g, String src,
                                   Map<String, String> prev) {
    Map<String, Integer> dist = new HashMap<>();
    g.keySet().forEach(n -> dist.put(n, Integer.MAX_VALUE));
    dist.put(src, 0);
    var pq = new PriorityQueue<PQ>(Comparator.comparingInt(PQ::cost));
    pq.add(new PQ(0, src));
    while (!pq.isEmpty()) {
        var top = pq.poll();
        if (top.cost() > dist.get(top.node())) continue; // stale entry - skip
        for (Edge e : g.get(top.node())) {
            int nc = top.cost() + e.w();
            if (nc < dist.get(e.to())) { // relax
                dist.put(e.to(), nc);
                prev.put(e.to(), top.node());
                pq.add(new PQ(nc, e.to()));
            }
        }
    }
    return dist;
}
// path: follow prev[] from target back to source, then reverse.

```

DIJKSTRA + PATH RECONSTRUCTION

C++

```

map<string, int> dijkstra(map<string, vector<pair<string,int>>>& g, string src,
                        map<string, string>& prev) {
    map<string, int> dist;
    for (auto& [n, _] : g) dist[n] = INT_MAX;
    dist[src] = 0;
    priority_queue<pair<int,string>, vector<pair<int,string>>, greater<>> pq;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [cost, node] = pq.top(); pq.pop();
        if (cost > dist[node]) continue; // stale entry - skip
        for (auto& [nb, w] : g[node]) {
            int nc = cost + w;
            if (nc < dist[nb]) { // relax
                dist[nb] = nc;
                prev[nb] = node;
                pq.push({nc, nb});
            }
        }
    }
    return dist;
}
// path: follow prev[] from target back to source, then reverse.

```

Full trace — Dijkstra from Mumbai

Step	Expanded	Delhi	Hyderabad	Chennai	Bangalore	What happened
init	—	∞	∞	∞	∞	dist[Mumbai]=0
1	Mumbai (0)	5	3	∞	∞	relax both links
2	Hyderabad (3)	5	3	∞	5	$3+2=5 < \infty \rightarrow$ improve B
3	Delhi (5)	5	3	13	5	$5+8=13 \rightarrow$ improve C
4	Bangalore (5)	5	3	9	5	$5+4=9 < 13 \rightarrow$ improve C
5	Chennai (9)	5	3	9	5	nothing improves — done

Final shortest paths from Mumbai:

→ Delhi: 5 (direct)
→ Hyderabad: 3 (direct)
→ Bangalore: 5 (Mumbai → Hyderabad → Bangalore)
→ Chennai: 9 (Mumbai → Hyderabad → Bangalore → Chennai)

NUMERICAL Reproduce this exact table in the exam: one row per expansion, always expand the current minimum, update only when strictly smaller. Chennai improved twice ($13 \rightarrow 9$) — that's Dijkstra working.

5 BFS vs DFS vs Dijkstra

Feature	BFS	DFS	Dijkstra
Data structure	Queue (FIFO)	Stack / recursion	Min-heap
What it finds	Fewest hops	A path (any)	Minimum cost
Time complexity	$O(V + E)$	$O(V + E)$	$O((V+E) \log V)$
Uses weights?	No — ignores	No — ignores	Yes — required
Negative weights?	N/A	N/A	No — breaks (L6)
Network use	RIP (hop count)	Topology / loop detection	OSPF (link cost)

Algorithms → Real Protocols

Algorithm	Real protocol	Role
BFS (hop count)	RIP — Routing Information Protocol	metric = hop count
Dijkstra	OSPF — Open Shortest Path First	every router runs Dijkstra on the full topology
Bellman-Ford	RIP, BGP (conceptually)	distributed updates — next lecture
MST algorithms	STP — Spanning Tree Protocol	prevents Layer-2 loops

LECTURE 5 — RAPID REVISION CARD

Network = graph: routers V , links E , weight = cost.

Adjacency list for sparse real networks ($O(V+E)$);
matrix for dense/fast-lookup ($O(V^2)$).

BFS: queue $\rightarrow$ fewest hops $\rightarrow$ **RIP**. Explores in layers;
first arrival = shortest hop path.

DFS: stack $\rightarrow$ deep-first; cycle detection (back-edge) =
routing-loop finder; $O(V+E)$.

Dijkstra: min-heap, greedy, relax = "new cost < known?"; $O((V+E) \log V)$; **no negative weights**.

Mumbai example final costs: Delhi 5 · Hyd 3 · Bang 5 ·
Chennai 9.

Stale-heap-entry check: if `cost > dist[node]`:
continue.

Path reconstruction: follow `prev[]` from target back to
source, then reverse.

✦ Important Questions — Exam Practice

A. Multiple Choice

Q1. Which data structure distinguishes BFS from DFS?

MCQ

- a) Heap vs array
- b) Queue vs stack
- c) Set vs list
- d) Graph vs tree

Q2. Dijkstra fails on negative edge weights because it:

MCQ

- a) uses too much memory
- b) finalises a node before a cheaper path through a negative edge appears
- c) cannot use a heap
- d) only works on trees

Q3. In the Figure-1 network, the minimum-cost path Delhi $\rightarrow$ Bangalore is:

MCQ

- a) Delhi $\rightarrow$ Hyderabad $\rightarrow$ Bangalore = 4
- b) Delhi $\rightarrow$ Mumbai $\rightarrow$ Hyderabad $\rightarrow$ Bangalore = 10
- c) Delhi $\rightarrow$ Chennai $\rightarrow$ Bangalore = 12
- d) direct — no such link

Q4. Why is an adjacency list preferred for Internet-scale routing?

MCQ

- a) It sorts edges automatically
- b) The Internet is sparse — $O(V+E)$ storage beats $O(V^2)$
- c) Matrices cannot store weights
- d) Lists enable BFS only

Q5. Which real protocol runs Dijkstra on a full topology map?

MCQ

- a) RIP
- b) OSPF
- c) ARP
- d) DHCP

B. Short Answer (2–4 marks)

Q6. Model a network as a weighted graph: what are vertices, edges, weights? Give two realistic weight choices.

SHORT

Q7. When would you choose an adjacency matrix over a list? What is the space complexity of each?

SHORT

Q8. Why can't BFS give minimum-cost paths? Give the concrete 1-hop vs 3-hop example.

SHORT

Q9. How does DFS detect a cycle? What does a cycle mean in a routing graph?

SHORT

Q10. Why does Dijkstra's code check `if cost > dist[node]: continue`?

SHORT

C. Long Answer / Numericals (5–8 marks)

Q11. Trace BFS from **Chennai** on Figure 1: give the visit order and the hop-distance of every node.

NUMERICAL

Q12. Run Dijkstra from **Delhi** on Figure 1. Show the table (one row per expansion), final costs to all nodes, and the shortest path Delhi → Chennai.

NUMERICAL

Q13. Write Dijkstra's pseudocode and explain the role of the min-heap, the relaxation step, and the `prev[]` map. State time complexity.

LONG

Q14. "Routing protocols are graph algorithms in disguise." Justify with the BFS↔RIP and Dijkstra↔OSPF mappings, and explain what a routing loop is using DFS cycle language.

LONG

ANSWER KEY

A1. b — queue (FIFO) vs stack (LIFO).

A2. b — greedy finalisation is wrong when a longer route later becomes cheaper.

A3. a — $2 + 2 = 4$ beats 10 and 12.

A4. b — sparse → list is compact and fast.

A5. b — OSPF; RIP is BFS-like hop count.

A6. V = routers, E = links, weight = cost. Realistic: latency (ms) or bandwidth-derived cost (Lecture 8).

A7. Dense graphs / constant-time edge lookup. List $O(V+E)$; matrix $O(V^2)$.

A8. BFS ignores weights — 1 hop costing 100 "wins" over 3 hops costing 3.

A9. A back-edge to a node in the current recursion stack. In routing = packet-loop risk.

A10. Lazy deletion: a better route may have popped first; stale heap entries must be skipped.

A11. Order: Chennai → Delhi → Bangalore → Mumbai → Hyderabad. Hops: Delhi 1, Bangalore 1, Mumbai 2, Hyderabad 2.

A12. Expansions: Delhi(0) → Mumbai(5), Hyderabad(2) → Bangalore(4) → Chennai(8)... full: Delhi 0, Hyderabad 2, Mumbai 5, Bangalore 4, Chennai 8; Delhi→Chennai = Delhi → Hyderabad → Bangalore → Chennai (cost 8).

A13. See code block; heap = next-cheapest node in $O(\log V)$; relaxation = improve dist via neighbour; `prev[]` = rebuild path backwards.

A14. RIP = distributed BFS (hop metric); OSPF = every router runs Dijkstra on the LSDB; a routing loop is exactly a cycle packets can traverse — detected/avoided via topology knowledge.